TRIAD: A Relational Model of Clinical Assertions*

Asish A. George
University of Victoria
Victoria, BC, Canada
asishgeorge@uvic.ca

Mohamed Khaled
University of Victoria
Victoria, BC, Canada
mabuela@uvic.ca

ABSTRACT

Clinical records are written by many people over years, and their facts change, yet a chart that stores only values cannot tell a correction from a contradiction. We map the TRIAD entity-relationship (ER) model to 29 relations in SQLite. Every clinical fact is an assertion carried by a message, issued by a staff member or the patient, and linked to the earlier assertion it supersedes, so the current fact is the one nothing supersedes. Keys and 11 triggers enforce this: assertions are append-only, and each entry has one current assertion. A web application demonstrates retrieval, insertion, update and deletion on Synthea data. Code: https://github.com/MohamedKhalidmk/TRIAD.

KEYWORDS

Clinical records, relational data modeling, SQLite, SQL, provenance, data integrity

1 PROBLEM DEFINITION

Physicians spend about 49% of their office day on records and desk work [3], much of it piecing together facts that changed. In 2019 a patient tells the front desk she had a rash after penicillin as a child, and the medical office assistant (MOA) records a penicillin allergy. In 2024 another clinic's record is copied in, repeating it. In 2026 an allergy test is negative, and the doctor rules the allergy out. A chart that stores only values still shows the penicillin banner: the correction looks like a contradiction, and a person, or an artificial intelligence (AI) assistant, must read every note to find what is true now.

Information need. For every clinical entry, return the current assertion (the one nothing supersedes), who issued it, through which message and when, and the chain of assertions it replaced. The schema must enforce this with keys and triggers, not by convention.

2 PROPOSED RELATIONAL MODEL

The ER diagram, revised after the Milestone 2 feedback (richer attributes, Staff *issues* Message, Assertion weak on Message, Visit no longer weak), maps to 29 relations using the course's ER-to-relational rules. Every relation has an autogenerated `Id`; every ER key becomes a mandatory candidate key (CK), usually the Fast Healthcare Interoperability Resources (FHIR) [2] identifier pair. All relations are listed on page 3.

- **Multivalued attributes** become relations of their own, e.g. `StaffLanguage`.
- **Many-to-one** relationships put the foreign key (FK) on the many side, e.g. `Visit.PatientId`, mandatory.
- **Many-to-many** relationships become relations with both FKs: `WorksIn` and `VisitLocation`.
- **One-to-one** *Supersedes* is a nullable FK, `SupersedesId` on `Assertion`, made a CK, so no assertion is superseded twice.
- **Weak entities**: `Assertion` is owned by `Message`, with CK (`MessageId`, `Seq`).
- **IS-A** uses one relation per parent and per child (Staff: Doctor, Nurse, MOA; ClinicalEntry: eight subtypes). The child's FK is the pair (`StaffId`, `Role`) with a CHECK on `Role`, so a Nurse row cannot point at a Doctor.

The core. A staff member issues a `Message`; a message from the patient has no issuer (`Origin = 'Patient'`). A message contains assertions; each is about one `ClinicalEntry` and may supersede one earlier assertion. Triggers enforce what keys cannot: assertions are append-only, a superseded assertion is older and about the same patient, and each entry has at most one current assertion.

3 RELATIONAL OPERATIONS

Structured query language (SQL) script `01_create_tables.sql` creates the 29 tables and 11 triggers; `02_insert_seed.sql` fills every table. A loader also imports the 112 Synthea [4] patients and our curated FHIR bundle (113 patients in all, 5,090 visits, 57,022 assertions). Foreign keys are enforced with `PRAGMA foreign_keys = ON`. The query numbers are those of `03_select_queries.sql`.

3.1 SELECT

Q1, what is true now, returns every assertion that nothing supersedes, with its entry and author (8 rows for the seed patient):

```
SELECT e.Category, e.Display, a.Verification,
       COALESCE(s.Name, 'Patient') AS Author, a.Statement
FROM Patient p JOIN Visit v ON v.PatientId = p.Id
JOIN ClinicalEntry e ON e.VisitId = v.Id
JOIN Assertion a ON a.ClinicalEntryId = e.Id
JOIN Message m ON m.Id = a.MessageId LEFT JOIN Staff s ON s.Id = m.StaffId
WHERE p.Name = 'Elena Marsh' AND p.DateOfBirth = '1971-05-14'
  AND NOT EXISTS (SELECT 1 FROM Assertion n WHERE n.SupersedesId = a.Id);
```

Q2, the revision chain, follows *Supersedes* back from the current penicillin assertion: three rows, 2026, 2024, then 2019.

```
WITH RECURSIVE chain (Id, SupersedesId, Depth) AS (
  SELECT a.Id, a.SupersedesId, 0 FROM Assertion a
  JOIN Allergy al ON al.ClinicalEntryId = a.ClinicalEntryId
  JOIN ClinicalEntry e ON e.Id = a.ClinicalEntryId
  JOIN Visit v ON v.Id = e.VisitId JOIN Patient p ON p.Id = v.PatientId
  WHERE p.Name = 'Elena Marsh' AND p.DateOfBirth = '1971-05-14'
    AND lower(al.Substance) LIKE '%penicillin%'
    AND NOT EXISTS (SELECT 1 FROM Assertion n WHERE n.SupersedesId = a.Id)
  UNION ALL
  SELECT a.Id, a.SupersedesId, c.Depth + 1
  FROM Assertion a JOIN chain c ON a.Id = c.SupersedesId)
SELECT c.Depth, date(a.AssertedAt), a.Verification, a.Statement
FROM chain c JOIN Assertion a ON a.Id = c.Id ORDER BY c.Depth;
```

Q8 aggregates over the IS-A: staff per department and role (13 rows; Cardiology has one doctor, one MOA and three nurses).

*Code, data, setup instructions, license and AI review log: https://github.com/MohamedKhalidmk/TRIAD.

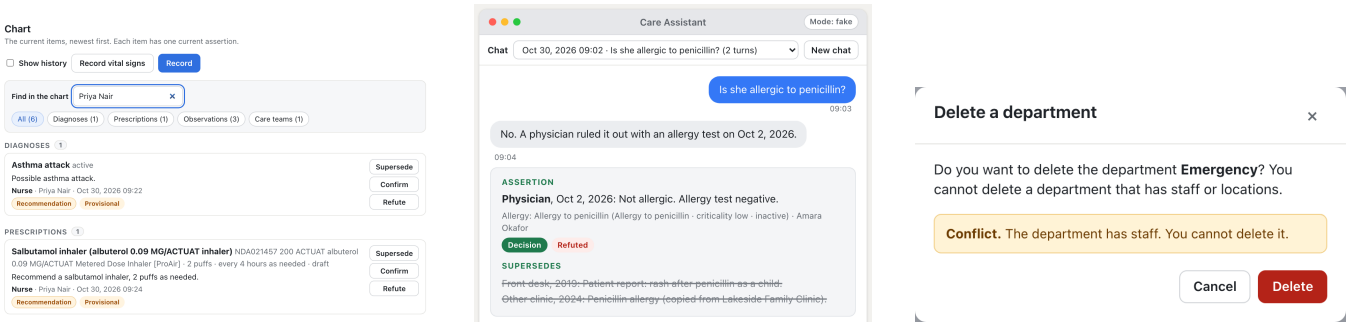


Figure 1: User interface. Left: a chart search finds the nurse's triage records. Middle: the penicillin answer cites the doctor's 2026 assertion and the superseded 2019 and 2024 claims (Q2). Right: deleting a department that has staff is refused.

```
SELECT d.Name, s.Role, COUNT(*) AS Staff,
       SUM(n.Designation = 'RN') AS RegisteredNurses
FROM Department d JOIN WorksIn w ON w.DepartmentId = d.Id
JOIN Staff s ON s.Id = w.StaffId LEFT JOIN Nurse n ON n.StaffId = s.Id
GROUP BY d.Id, s.Role;
```

3.2 INSERT, UPDATE and DELETE

An update and an FK-safe delete:

```
UPDATE Patient SET PhoneNumber = '+1-508-555-0199'
WHERE Name = 'Elena Marsh' AND DateOfBirth = '1971-05-14';
DELETE FROM Location WHERE IdentifierValue = 'STORE-OLD'
  AND NOT EXISTS (SELECT 1 FROM VisitLocation vl
                  WHERE vl.LocationId = Location.Id);
```

A clinical fact is never overwritten. A patient report through the app inserts a `Message` with no issuer and an `Assertion` that supersedes the current one; the old one stays (script 04, step 5):

```
INSERT INTO Message (IdentifierSystem, IdentifierValue, StaffId, Channel,
  Origin, SentAt, Text) VALUES ('https://triad.example/fhir/sid/message',
  'MSG-ELENA-APP-0001', NULL, 'App', 'Patient', '2026-10-30T08:30:00-04:00',
  'I take amlodipine every morning.');
INSERT INTO Assertion (MessageId, Seq, ClinicalEntryId, SupersedesId,
  AssertedAt, Intent, Source, Verification, Statement)
SELECT last_insert_rowid(), 1, a.ClinicalEntryId, a.Id,
  '2026-10-30T08:30:00-04:00', 'Report', 'Human', 'Unconfirmed',
  'I take amlodipine every morning.'
FROM Assertion a JOIN ClinicalEntry e ON e.Id = a.ClinicalEntryId
WHERE e.IdentifierValue = 'MedicationRequest/mr-elena-amlodipine'
  AND NOT EXISTS (SELECT 1 FROM Assertion n WHERE n.SupersedesId = a.Id);
```

Changing an assertion in place fails (*Assertion is append-only*), as do the 26 test cases of `05_constraint_violations.sql`.

4 ARCHITECTURE AND USER INTERFACE

Figure 2 shows three layers. In the *user interface*, a React application sends JavaScript Object Notation (JSON) requests to a FastAPI application programming interface (API); each request names the acting user (doctor, nurse, MOA or patient). The *logic* layer holds the domain rules (supersede, current state, what a patient may see) and an access check: only the patient's care team may read a chart, and a denied attempt is written to `AuditEvent`. The *data* layer issues plain SQL to SQLite. Figure 1 shows the operations.

5 REPRODUCIBILITY AND EVALUATION

`src/scripts/quickstart.sh` checks the prerequisites, installs pinned packages, builds the frontend and the database, runs the fast tests and serves the application; the chat uses a fake large language model (LLM) by default, so no key or network is needed. On a fresh

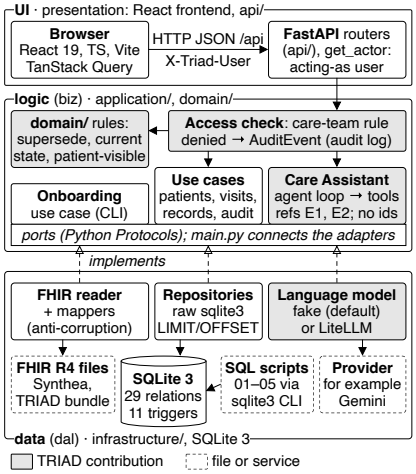


Figure 2: Architecture. Grey boxes are TRIAD's contribution.

clone, the application served in about 25 s, the five scripts ran with no FK violations, and the full suite of 422 tests passed.

AI use and critique. Claude Code [1] generated most of the code, SQL and tests; `doc/ai_review_log.md` records what we checked. The AI dropped our no-loops rule in the ER diagram to mirror FHIR, proposed one visit per version of a fact (we rejected it), and wrote code that passed type checks yet silently ignored a phone update; only an end-to-end test caught it. These errors fall where the model decided alone. A language model recognizes and continues patterns: given our design and examples, it was reliable; asked to decide without them, it guessed. We treat it as a junior engineer, not the architect. Claude also helped draft this report; we checked every number and query.

REFERENCES

[1] Anthropic. 2026. Claude Code. https://claude.com/claude-code.
[2] HL7 International. 2019. HL7 FHIR Release 4 (v4.0.1). https://hl7.org/fhir/R4/.
[3] Christine Sinsky et al. 2016. Allocation of Physician Time in Ambulatory Practice: A Time and Motion Study in 4 Specialties. *Annals of Internal Medicine* 165, 11 (2016), 753–760.
[4] Jason Walonoski et al. 2018. Synthea: An Approach, Method, and Software Mechanism for Generating Synthetic Patients and the Synthetic Electronic Health Care Record. *Journal of the American Medical Informatics Association* 25, 3 (2018), 230–238.

RELATIONS AND CONSTRAINTS

Every relation has an autogenerated surrogate key Id (INTEGER PRIMARY KEY AUTOINCREMENT). CK = candidate key, enforced by UNIQUE and NOT NULL; every ER key becomes a CK. FK = foreign key to the target's Id, with no cascading delete. ? marks an optional (nullable) attribute; all others are mandatory. Ident = (IdentifierSystem, IdentifierValue), the FHIR identifier. Addr = Street?, City?, Region?, PostalCode?, Country?. "As in 14" = CK ClinicalEntryId; FK (ClinicalEntryId, Category) → ClinicalEntry(Id, Category).

(1) Department(Id, Ident, Name, Type?, PhoneNumber?, Email?, Addr). CK Ident.
(2) Location(Id, Ident, Name, Type?, PhysicalType?, PhoneNumber?, Addr). CK Ident.
(3) Staff(Id, Ident, Name, Sex?, Email?, PhoneNumber?, Active, Role). CK Ident; CK (Id, Role) for the subtypes. Role ∈ {Doctor, Nurse, MOA}; Active ∈ {0, 1}.
(4) StaffLanguage(Id, StaffId, Language). CK (StaffId, Language); FK StaffId → Staff.
(5) WorksIn(Id, StaffId, DepartmentId, RoleCode, Specialty?, StartDate?, EndDate?). CK (StaffId, DepartmentId, RoleCode); FK StaffId → Staff; FK DepartmentId → Department. EndDate ≥ StartDate.
(6) Doctor(Id, StaffId, Role, LicenseNumber?, Qualification?). CK StaffId; FK (StaffId, Role) → Staff(Id, Role). Role = Doctor.
(7) Nurse(Id, StaffId, Role, LicenseNumber?, Designation). CK StaffId; FK (StaffId, Role) → Staff(Id, Role). Role = Nurse; Designation ∈ {RN, LPN, RPN, NP}.
(8) MOA(Id, StaffId, Role, CertificationNumber?, Desk, PhoneExtension?). CK StaffId; FK (StaffId, Role) → Staff(Id, Role). Role = MOA.
(9) Patient(Id, Name, DateOfBirth, Sex, PhoneNumber?, Addr). CK (Name, DateOfBirth). Sex ∈ {female, male, other, unknown}.
(10) PatientIdentifier(Id, PatientId, System, Value). CK (System, Value); FK PatientId → Patient.
(11) Visit(Id, Ident, PatientId, Class, Type?, Reason?, StartDate, EndDate?). CK Ident; FK PatientId → Patient (Has). Class ∈ {AMB, EMER, IMP, HH, VR}; EndDate ≥ StartDate.
(12) VisitLocation(Id, VisitId, LocationId, StartDate, EndDate?). CK (VisitId, LocationId, StartDate); FK VisitId → Visit; FK LocationId → Location (TakesPlaceIn). EndDate ≥ StartDate.
(13) ClinicalEntry(Id, Ident, VisitId, Category, Code, CodeSystem, Display, Text?, RecordedAt). CK Ident; CK (Id, Category) for the subtypes; FK VisitId → Visit (During, total). Category is one of the eight subtypes.
(14) Diagnosis(Id, ClinicalEntryId, Category, OnsetDate?, AbatementDate?, ClinicalStatus, Severity?). CK ClinicalEntryId; FK (ClinicalEntryId, Category) → ClinicalEntry(Id, Category). Category = Diagnosis.
(15) Procedure(Id, ClinicalEntryId, Category, PerformedDate, Outcome?). As in 14; Category = Procedure.
(16) Allergy(Id, ClinicalEntryId, Category, Substance, Type?, AllergyCategory?, Criticality?, ClinicalStatus?). As in 14; Category = Allergy.
(17) Observation(Id, ClinicalEntryId, Category, Value, Unit?). As in 14; Category = Observation.
(18) LabOrder(Id, ClinicalEntryId, Category, TestName, Priority, Specimen?). As in 14; Priority ∈ {routine, urgent, asap, stat}.
(19) LabReport(Id, ClinicalEntryId, Category, IssuedDate, Conclusion?). As in 14; Category = LabReport.
(20) Medication(Id, Code, CodeSystem, Name, Form?, Manufacturer?). CK (Code, CodeSystem).
(21) Prescription(Id, ClinicalEntryId, Category, MedicationId, Dosage?, Frequency?, Status). As in 14; FK MedicationId → Medication (Prescribes).
(22) CareTeam(Id, ClinicalEntryId, Category, Name?, Status). As in 14; Status ∈ {proposed, active, suspended, inactive, entered-in-error}.
(23) CareTeamMember(Id, CareTeamId, MemberSystem, MemberValue, Role?). CK (CareTeamId, MemberSystem, MemberValue); FK CareTeamId → CareTeam. The member must be a staff identifier (trigger).
(24) Message(Id, Ident, StaffId?, Channel, Origin, SentAt, Text). CK Ident; FK StaffId → Staff (Issues), optional. StaffId IS NOT NULL OR Origin = Patient.
(25) Assertion(Id, MessageId, Seq, ClinicalEntryId, SupersedesId?, AssertedAt, Intent, Source, Verification, Statement). CK (MessageId, Seq) (weak entity); CK SupersedesId (optional, one-to-one); FK MessageId → Message (Contains); FK ClinicalEntryId → ClinicalEntry (About); FK SupersedesId → Assertion (Supersedes). SupersedesId ≠ Id; Intent, Source and Verification take fixed value sets.
(26) ChatSession(Id, Ident, UserRole, UserSystem, UserValue, SubjectName, SubjectDateOfBirth, StartedAt, Mode, Model?). CK Ident. Mode ∈ {rules, fake, llm}; user and subject are stored as values.
(27) ChatTurn(Id, ChatSessionId, Seq, ClientTurnId, Speaker, Text, CreatedAt). CK (ChatSessionId, Seq) (weak entity); CK (ChatSessionId, ClientTurnId); FK ChatSessionId → ChatSession (Includes). Speaker ∈ {User, Assistant}.
(28) Citation(Id, ChatTurnId, AssertionId, Rank). CK (ChatTurnId, AssertionId); FK ChatTurnId → ChatTurn; FK AssertionId → Assertion (Cites). Rank ≥ 1.
(29) AuditEvent(Id, Ident, At, Action, Outcome, ActorRole, ActorSystem, ActorValue, SubjectName, SubjectDateOfBirth, Path, ChatTurnId?). CK Ident; FK ChatTurnId → ChatTurn (InTurn), optional. Outcome ∈ {Allowed, Denied}; actor and subject are stored as values.

Triggers (11 rules a CHECK cannot express). Assertions are append-only: no UPDATE or DELETE; a correction is a new superseding assertion. A superseded assertion must be older and about the same patient. Each entry has at most one current assertion. All assertions in one message concern one patient. A care-team member must match a staff identifier. Only an assistant turn may cite, and only assertions about the chat's patient. Audit records are append-only, and an audit record tied to a chat turn must name that chat's patient. A patient's key (Name, DateOfBirth) cannot be updated, since the logs store it as a value.

Mapping assumptions. Patients are identified by (Name, DateOfBirth), with external record numbers in PatientIdentifier. A message without a staff issuer comes from the patient. Chat and audit records keep the user and patient as values, so they stay correct when a row changes and add no loop to the ER diagram. Times are ISO 8601 text with an offset from Coordinated Universal Time (UTC).